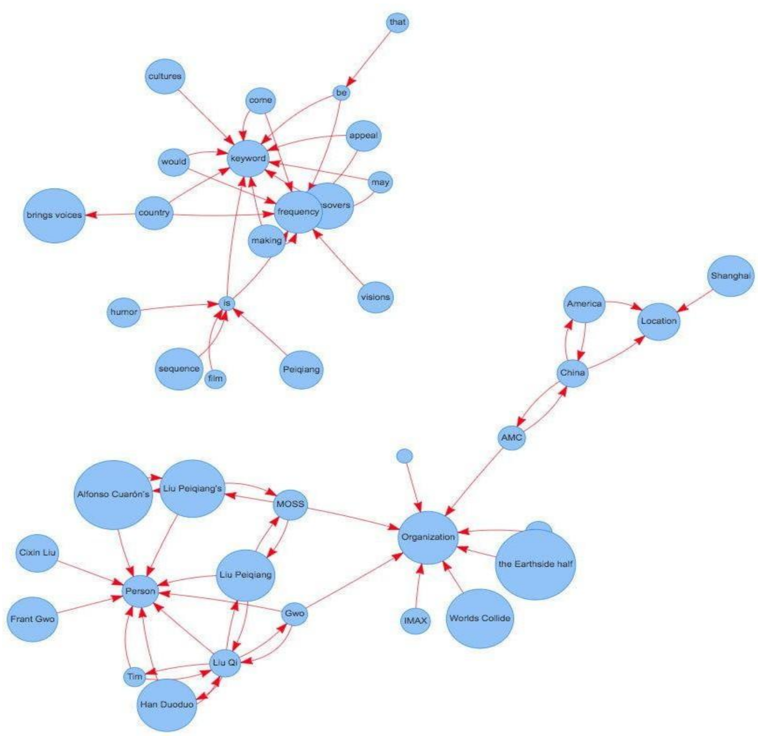
Aim: Representation of a document with their keywords using TextRank

IDE: Google Colab

Theory:

TextRank is an algorithm based on PageRank, which often used in keyword extraction and text summarization. In this article, I will help you understand how TextRank works with a keyword extraction example and show the implementation by Python.

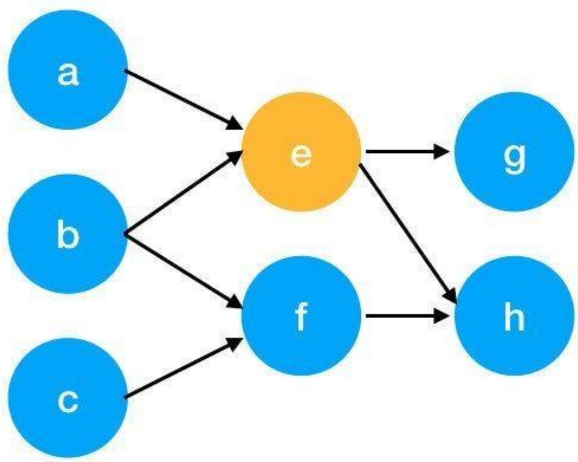


Understand PageRank

There are tons of articles talking about PageRank, so I just give a brief introduction to PageRank. This will help us understand TextRank later because it is based on PageRank. PageRank (PR) is an algorithm used to calculate the weight for web pages. We can take all web pages as a big directed graph. In this graph, a node is a webpage. If webpage A has the link to web page B, it can be represented as a directed edge from A to B. After we construct the whole graph, we can assign weights for web pages by the following formula.

$$S(V_i) = (1-d) + d * \sum_{j \in In(v_i)} \frac{1}{|Out(V_j)|} S(V_j)$$

- S(Vi) - the weight of webpage i
- d - damping factor, in case of no outgoing links
- In(Vi) - inbound links of i, which is a set
- Out(Vj) - outgoing links of j, which is a set
- |Out(Vj)| - the number of outbound links



Here is an example to better understand the notation above. We have a graph to represent how web pages link to each other. Each node represents a webpage, and the arrows represent edges. We want to get the weight of webpage e. We can rewrite the summation part in the above function to a simpler version.

$$In(v_e) = \{a, b\}, j \in \{a, b\}$$

$$\begin{aligned}\sum_{j \in \{a,b\}} \frac{1}{|Out(V_j)|} S(V_j) &= \frac{1}{|Out(V_a)|} S(V_a) + \frac{1}{|Out(V_b)|} S(V_b) \\ &= \frac{1}{|\{e\}|} S(V_a) + \frac{1}{|\{e, f\}|} S(V_b) \\ &= S(V_a) + \frac{1}{2} S(V_b)\end{aligned}$$

We can get the weight of webpage e by the following function.

$$S(V_e) = (1 - d) + d * \left(S(V_a) + \frac{1}{2} S(V_b) \right)$$

We can see the weight of the webpage e is dependent on the weights of its inbound pages. We need to run this iteration much time to get the final weight. In the initialization, the importance of each webpage is 1.

Pre Lab Exercise:

1. What are the key components of TextRank?

Ans: textrank is based on graph based ranking using pagerank algorithm words or sentences are treated as nodes and their relationship as edges, it uses co occurrence or words to build the graph, importance of nodes is calculated iteratively.

2. What are some limitations of TextRank for keyword extraction?

Ans: textrank does not understand semantic meaning of words, it depends heavily on co occurrence frequency, may extract irrelevant or duplicate keywords, performance depends on quality of preprocessing.

3. What are the advantages of TextRank for keyword extraction?

Ans: it is unsupervised and does not require labeled data, simple and effective for keyword extraction, works well for large text documents, language independent and easy to implement.

Program (Code):

```
#lab 9


# Step 1: Install Libraries
!pip install summa
```

```
# Step 2: Import Library
from summa import keywords
```

```
# Step 3: Input Text
text = """TextRank is an algorithm used for keyword extraction and summarization.
It is based on the PageRank algorithm and helps in identifying important words
from a document by analyzing word relationships and co-occurrence patterns."""
```

```
# Step 4: Extract Keywords
key_words = keywords.keywords(text, words=10)
```

```
print("Extracted Keywords:\n")
print(key_words)
```

Results:

```
Extracted Keywords:

algorithm
pagerank
identifying important words
analyzing word relationships
patterns
extraction
```

Observation:
Textrank extracts keywords based on word importance and relationships, it identifies significant words without requiring training data, extracted keywords represent the main idea of the document, some irrelevant or repeated keywords may also appear, it is effective for summarization and keyword extraction.

Post Lab Exercise:

1. Write about "your ownself" (in not more than 500 words→ You know better about you, rather than ChatGPT!!). Extract top-25 keywords for your portfolio and analyze it in 3 categories: keyword can not give the real idea, keyword gives the real idea, keyword is irrelevant. Paste the code, your portfolio, output and your analysis.

Code:

```
# Step 1: Import Library
from summa import keywords

# Step 2: Write your own portfolio text
portfolio = """I am a Computer Engineering student currently pursuing my B.Tech.
```

```
I have a strong interest in Artificial Intelligence, Machine Learning, and Web Development.
I have worked on projects like chatbot systems, stock prediction using LSTM, and IoT-based systems.
I enjoy solving real-world problems and participating in hackathons.
I have also won competitions and actively contribute to technical communities."""
```

```
# Step 3: Extract Top 25 Keywords
key_words = keywords.keywords(portfolio, words=25)
```

```
print("Top Keywords:\n")
print(key_words)
```

Output:

```
Top Keywords:

engineering student currently pursuing
communities
technical
actively contribute
development
web
intelligence
machine
won competitions
artificial
learning
systems stock prediction
projects like chatbot
iot
```